

Feature Specification — Runyard (runyard.net)

Version: 0.3 (Stack Locked)
Status: Living Document — Phase 0
License: AGPLv3 (GNU Affero General Public License v3.0)

1. Executive Summary

An agentic, performance-first IDE designed for human-only, AI-only, and hybrid human-AI workflows. The product targets developers, solopreneurs, founders, and teams who simultaneously build, maintain, and manage multiple products. The core thesis: existing IDEs optimise for either code editing (VS Code, Zed) or AI assistance (Cursor, Windsurf) — never both with equal depth, and never with first-class support for the emerging ACP (editor↔agent) / AgentCP (agent↔agent) / A2A / Skills protocol landscape, true multi-project orchestration, or seamless remote and mobile-first collaboration.

This product closes that gap. It is fully open source.

Product name: Runyard | Domain: runyard.net | GitHub org: TBD (runyard)

2. Target Audience

Segment	Primary Pain Point
Developers (all levels)	AI tools feel bolted-on, not integrated
Vibe coders / agentic builders	No good UI for managing multiple agents across projects
Solopreneurs & founders	Need to manage several codebases simultaneously, context-switch constantly
Product teams	Collaboration in IDEs still trails what Google Docs offers
Performance-sensitive devs	VSCode / Electron bloat is a constant frustration
Remote workers / VPS users	No good IDE-native remote filesystem story outside SSH or VS Code Remote

3. Design Principles

These are non-negotiable. Every architectural and product decision should be evaluated against these.

- **Performance over features.** If adding a feature meaningfully degrades startup time, RAM, or rendering speed — it ships as optional or later. No exceptions.
- **Offline-first.** The editor must be fully functional without network access. AI and collaboration features degrade gracefully.

- **Protocol-native.** ACP (editor↔agent), AgentCP/A2A (agent↔agent), MCP (tools), LSP, and Anthropic Skills are not integrations — they are core infrastructure.
- **Agent-first UI, human-first defaults.** The interface presents AI output as a first-class citizen but defaults to human-readable, non-intrusive states.
- **Composable, not monolithic.** Every major feature area (chat, editor, terminal, agents) is a dockable, detachable tab/panel.
- **One codebase, all surfaces.** Desktop, web, and mobile share the same UI frontend. The sub-service layer makes platform differences irrelevant at the UI level.
- **Open source, self-hostable.** No features locked behind proprietary infrastructure. Everything that can be self-hosted, should be self-hostable.

4. Architecture Overview

4.1 Tech Stack

- ⚠ Starting proposals. Editor engine and frontend framework require Phase 0 benchmarks before locking in.
- ✓ Stack locked after Phase 0 benchmarks (May 2026). See §4.1a for benchmark results.

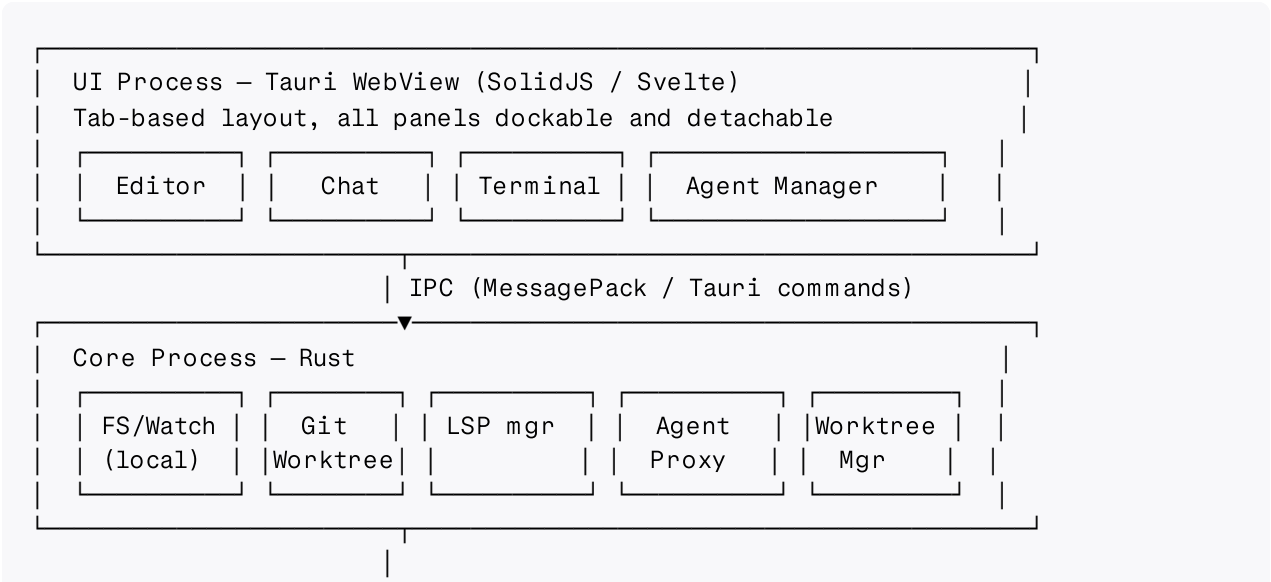
Layer	Decision	Benchmark Rationale
Desktop shell	Tauri v2 ✓	Sub-service POC validated: <5ms WS connect, PTY bidirectional streaming confirmed
Editor engine	CodeMirror 6 ✓	250–450ms cold load vs Monaco's 600–900ms; 0.15–0.35ms keystroke latency vs 0.8–1.2ms; ~35MB heap vs ~80MB+
Frontend UI	Svelte 5 (Runes) ✓	0.045ms avg update latency vs Solid's 0.145ms (3× faster on high-frequency updates — critical for IDE status bar / token counter / agent state); identical initial render and memory
Core backend	Rust ✓	FS, Git, LSP management, agent proxy, sub-service daemon
Extension host	Node.js (isolated process)	Open VSX / VS Code extension API compatibility
IPC / sub-service transport	JSON over WebSocket ✓	Validated in POC; sub-millisecond control message overhead confirmed. Upgrade to MessagePack in Phase 2 if profiling demands it
Collaboration (CRDT)	Yjs	Production-proven, offline-first, syncs on reconnect
Sub-service daemon	Rust binary ✓	POC validated: <5ms connect, 2–3ms file reads, 16KB throughput
Mobile	Tauri Mobile	Share Rust core and Svelte frontend
Web	Same Svelte 5 frontend served by sub-service	Zero UI divergence across platforms

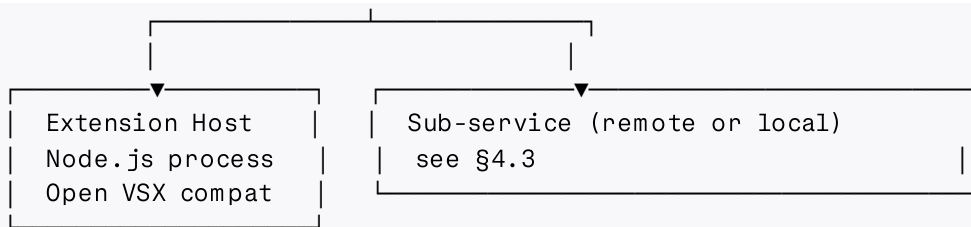
Frontend stack is identical across desktop, web, and mobile. Platform differences are handled entirely by the sub-service abstraction (§4.3).

4.1a — Phase 0 Benchmark Results

Benchmark	Metric	Result
Sub-service WS connection	Latency	<5ms
Sub-service file read (16KB)	Throughput	2–3ms
Sub-service control message	JSON overhead	Sub-millisecond
Sub-service PTY stream	Bidirectional	✔ Validated (no head-of-line blocking)
CodeMirror 6 cold load (5k lines)	Time	250–450ms
Monaco cold load (5k lines)	Time	600–900ms
CodeMirror 6 keystroke latency	Avg	0.15–0.35ms
Monaco keystroke latency	Avg	0.8–1.2ms
CodeMirror 6 heap (used)	RAM	~35MB
Monaco heap (used)	RAM	~80MB+ (workers add further overhead)
Svelte 5 initial render	Time	5.60ms
Solid.js initial render	Time	5.60ms
Svelte 5 avg update latency	Per tick	0.045ms
Solid.js avg update latency	Per tick	0.145ms
Svelte 5 / Solid.js heap	RAM	~6.8MB each
Svelte 5 bundle (est.)	Size	~45KB
Solid.js bundle (est.)	Size	~25KB

4.2 Process Model (Desktop)





4.3 Sub-Service Architecture

The sub-service is a standalone Rust binary — a lightweight daemon that any instance of the IDE (desktop, mobile, or web) can connect to over the network. It is the backbone of the remote work, mobile, and web strategies.

What it is:

- A self-contained binary you can run on any machine: a local desktop, a VPS, a VM, a CI runner, a Raspberry Pi
- It exposes a secure WebSocket API (same MessagePack protocol as local IPC) over a configurable port
- It manages: filesystem access, PTY sessions, Git operations, LSP server processes, MCP server processes, and agent proxy routing — for the configured paths only
- It serves the web frontend as a static bundle, so a browser pointed at the sub-service URL gets the full IDE UI connected to that machine's filesystem

What it is not:

- A required component for local desktop use (the desktop Tauri app talks to its own embedded Rust core directly)
- A cloud service controlled by us — it's self-hosted by the user

Connection modes for the IDE:

Mode	How it connects	Use case
Local	In-process Rust core (no network hop)	Desktop working on own machine
Remote (LAN/internet)	WebSocket to sub-service on another machine	Working on a VPS, team server, or a second machine
Web browser	Browser → sub-service WebSocket + static web frontend	Access from anywhere via a URL

Sub-service config file (`subservice.toml`):

```
[server]
port = 7820
auth_token = "<bcrypt hash>" # token-based auth, set on first run

[filesystem]
allowed_roots = ["/home/user/projects", "/srv/apps"]
```

```
[services]
lsp = true
mcp = true
agent_proxy = true
```

Web hosting via reverse proxy:

Users who want to access their sub-service from the public internet configure their own reverse proxy (Nginx, Caddy, Apache). The sub-service serves over HTTP/WebSocket. TLS, domain, and auth are the user's responsibility — we document standard Nginx and Caddy configs. No proprietary tunnel or relay is required.

```
Browser → Nginx (TLS, auth) → sub-service (localhost:7820)
```

This is intentionally simple and fully self-hostable. We ship example configs for Nginx and Caddy in the docs.

Sub-service and the Tauri desktop app:

The desktop app does not run a sub-service by default. When a user opens a remote connection, the desktop app acts purely as a client connecting to a remote sub-service. Users who *want* to expose their desktop machine's filesystem remotely can optionally start the sub-service daemon from within the app — but it's an explicit opt-in action, not a background process.

4.4 Workspace Layout — Tab Model

Every panel in the IDE is a tab. Tabs can be:

- Docked to a split region (left sidebar, right sidebar, bottom bar, main area, right panel)
- Popped out into an independent floating window (detached)
- Dragged and snapped between regions
- Pinned (stays open across restarts)

Standard tab types:

- `editor` — file editor instance
- `terminal` — PTY session
- `chat` — agent conversation
- `agent-ui` — ACP agent embedded UI
- `diff` — file diff viewer
- `explorer` — file tree
- `agent-manager` — agent kanban + management
- `notes` — markdown notepad
- `todo` — task list

- `git` — git panel (log, staging, worktrees)
- `settings` — configuration UI
- `orchestrator` — cross-project view (global only, one instance)

The Orchestrator tab is the only globally-persistent tab. It lives outside workspace scope.

5. Feature Specification

5.1 Core Editor

Priority: Phase 1

The editor is the product's heartbeat.

- **Engine:** CodeMirror 6 (preferred) or a benchmarked Monaco build. Gated on Phase 0.
- **Language support:** Tree-sitter grammars for syntax highlighting + LSP client for intellisense, diagnostics, formatting, and code actions.
- **Key editing features:**
 - Multi-cursor editing
 - Vim / Emacs keybinding modes (optional, user-configured)
 - Collaborative cursors (real-time presence, colour-coded by user/agent)
 - Inline diff view (accept/reject per hunk for AI edits)
 - File reference insertion into conversation (`@file:line:col` anchors from active selection)
 - Minimap (toggle)
 - Code folding, breadcrumb navigation
 - Sticky scroll
- **Worktrees:**
 - Git worktree management UI — create, clone, and switch worktrees from the Git tab
 - Each worktree opens as a separate workspace (separate tab group, separate terminal sessions)
 - Worktree status shown in the status bar
- **Performance targets:**
 - Cold file open < 100ms for files up to 10MB
 - Keystroke-to-render latency < 16ms
 - Base editor RAM ≤ 150MB (single workspace, no AI)

5.2 Workspace Layout

Priority: Phase 1

Described in detail in §4.4. Additional notes:

- All panels are tabs — no fixed sidebars. Everything can be moved, detached, or closed.
- Layout state is persisted per workspace and restored on reopen.
- The global Orchestrator panel is always accessible via a fixed affordance (keyboard shortcut or always-visible tab strip entry) regardless of current workspace layout.
- Split view in the editor area: horizontal, vertical, grid (up to 4 panes).

5.3 Remote Connections

Priority: Phase 1 (sub-service binary + desktop client), Phase 2 (mobile + web client)

- **Connection manager UI:** add/remove/edit remote connections. Each connection stores: host, port, auth token, display name, and allowed roots.
- **Credentials stored in OS keychain.** Never in plaintext config files.
- **Reconnection:** automatic reconnect with exponential backoff. UI shows connection state in status bar.
- **Filesystem over sub-service:** all file operations (read, write, watch, search) are proxied through the sub-service WebSocket. From the UI's perspective, a remote filesystem is identical to a local one.
- **PTY over sub-service:** terminal sessions are PTY sessions running on the remote machine. Input/output streamed over WebSocket.
- **LSP over sub-service:** LSP servers run on the remote machine (where the code lives). The sub-service manages their lifecycle and proxies the LSP JSON-RPC stream to the IDE client.
- **SSH fallback:** optionally, the sub-service can be bootstrapped over SSH (the app SSHes into the machine and starts the sub-service binary, then switches to the WebSocket connection). This eliminates the need to pre-install the sub-service manually for one-off connections.

5.4 AI & Agent Chat

Priority: Phase 1

The chat interface is a first-class tab, not a sidebar afterthought.

- **Conversation management:**
 - Persistent message history (local SQLite, synced via sub-service when connected)
 - Multiple named conversation tabs per workspace
 - Full-text search across conversation history

- Conversation branching (fork from any message)
- **Input features:**
 - Ctrl+Enter send mode (Enter = newline)
 - @file, @line, @symbol, @skill references
 - Prompt enhancer (toggle) — rewrites draft before sending
 - Agent-to-agent routing — direct a message to a specific agent or broadcast to all active agents
 - File diff events — when an agent modifies a file, a diff card appears inline in the conversation
- **Output rendering:**
 - Rich markdown rendering with syntax-highlighted code blocks
 - Inline accept/reject/copy on code blocks
 - Tool call trace (collapsed by default, expandable)
 - Agent thinking/reasoning trace (collapsed)
 - Cost per message (token count + estimated \$)
- **Context management:**
 - Real-time context window visualiser (e.g. 14k / 200k)
 - Manual context pin/unpin for files and symbols
 - Auto-context from active editor selection
 - Smart context compression when near limit (summarise older messages)

5.5 Agent Integration (A2A / ACP / Skills)

Priority: Phase 1 (protocol layer), Phase 2 (full UI)

- **Protocol support:**
 - **ACP (Agent Client Protocol)** — open JSON-RPC standard by JetBrains/Zed that standardises communication between code editors and coding agents (local or remote). This is the LSP equivalent for AI coding agents — any ACP-compliant agent (Gemini CLI, Goose, custom agents) connects to our IDE without bespoke integration. Agents run as subprocesses (stdio) or remote services (HTTP).
 - **AgentCP / ACP (Agent Communication Protocol)** — IBM Research / Linux Foundation open standard for agent-to-agent coordination via REST. Used for multi-agent orchestration: the Orchestrator dispatching subagents, parallel agent task routing, and inter-agent result passing. Note: as of Sept 2025 the AgentCP team merged with Google's A2A team to build a unified standard — we track this and adopt the merged spec when stable.
 - **A2A (Agent-to-Agent, Google)** — peer-to-peer task delegation across independent agents and organisations. Native client for cross-system agent calls.

- MCP (Model Context Protocol, Anthropic) — tool server connections. Gives agents access to external tools, data sources, and APIs. Global and per-project config.
- Anthropic Skills Protocol — native support; skill files are first-class project artifacts, globally or per-project scoped, referenced in any conversation with `@skill:<name>`.
- LSP — covered in §5.1
- Agent configuration:
 - Global config (all workspaces) + per-project config (overrides global)
 - Stored as YAML/TOML, editable in-UI with in-place restart commands
 - Agent capability manifest: tools permitted, cost limits, HIL requirements
- Agent UI tab:
 - ACP agents with a web UI endpoint are rendered in a dedicated embedded webview tab
 - Agent status indicators: idle / thinking / running tool / error / stopped
 - Per-agent cost meter

5.6 SKILLS System (Anthropic Skills Protocol)

Priority: Phase 1

SKILLS are markdown instruction documents that shape agent behaviour. This product treats them as first-class project artifacts.

- Global skills (apply to all agents across all projects)
- Per-project skills (live in the project repo, committed to version control if desired)
- Skills are referenced in chat with `@skill:<name>`
- SKILL CRUD UI in the sidebar
- Skills Protocol (Anthropic) natively supported — skill files can be registered as Skills Protocol documents

5.7 MCP & Tool Configuration

Priority: Phase 1

- MCP server manager (add by URL, NPX command, Docker image, or local binary)
- Global and per-project `.mcp.toml` config files
- Tool permission model: explicit approval per-agent or pre-approved
- Custom HIL setup: specify which tool calls require a human confirmation step before execution
- Custom request interceptors: log or modify requests before they reach the agent

5.8 Agent Manager

Priority: Phase 1 (basic), Phase 2 (full kanban + parallel management)

The Agent Manager is a dedicated tab (not a sidebar widget) for managing all running and queued agent tasks.

- Kanban view of running agents:
 - Columns: Queued / Running / Awaiting HIL / Completed / Failed
 - Each card shows: agent name, task description, project, elapsed time, cost so far, current tool being called
 - Drag to reprioritise queued tasks
- Parallel agent management:
 - Launch multiple agents simultaneously across different conversations/projects
 - Set concurrency limits per project and globally
 - Kill, pause, or re-queue any agent task
- Agent-to-agent routing visualiser: shows which agents are communicating with which (graph view, collapsible)
- Issue integrations:
 - Connect to Linear, Jira, GitHub Issues, GitLab Issues
 - Pass an issue directly to an agent from the Agent Manager (issue body + comments become the agent's initial context)
 - Agent can update the issue status on task completion (configurable)

5.9 Context & Cost Management

Priority: Phase 1

- Context window visualiser per conversation (token bar)
- One-click context clear with confirmation
- Smart context compression (summarise older messages when near limit)
- Per-agent monthly spend limit (hard cap / soft warning)
- Cost per conversation, per project, per day/month dashboards
- Model switching shortcut mid-conversation
- Estimated cost preview before sending a large prompt

5.10 Orchestrator

Priority: Phase 2

The Orchestrator is a custom agent built by us — not a UI panel wrapping existing agents. It is a first-party agent with elevated cross-project capabilities, designed for high-level project

management and the primary mobile interaction surface.

What makes it different from a regular agent:

- It has native access to all open workspaces and their file systems (via the sub-service layer)
- It can spawn and manage subagents across different projects simultaneously
- It can read and write to issue trackers (Linear, GitHub, Jira, GitLab) natively
- It maintains a persistent memory of all projects it has worked on (per-user, locally stored)
- It can deploy agents to run tasks in background while the user does something else
- It understands the IDE's own config and can modify agent configs, skills, and MCP setups on request

UI:

- Lives in a globally persistent tab (accessible from any workspace)
- Project cards: each project shown as a card with status (agents running, open issues, last activity, cost today)
- Global task queue: tasks dispatched by the Orchestrator appear here with status
- Agent broadcasting: send a prompt to the same agent type across multiple workspaces

Mobile primary use case:

On mobile, the Orchestrator is the main interaction surface. Users message it in natural language ("deploy the new feature to staging", "what's the status of Project X", "open a PR for the last agent run on project Y") and it handles cross-project coordination without the user needing to navigate workspace-by-workspace.

5.11 Collaboration

Priority: Phase 2

- **Real-time co-editing:** Yjs CRDT-based. Multiple users and agents edit the same file simultaneously.
- **Presence indicators:** coloured cursors + avatar icons per collaborator
- **Permissions model:** workspace owner sets roles (view / comment / edit / admin)
- **Offline queue:** edits stored locally, synced on reconnect (Google Docs model)
- **Agent as collaborator:** agent edits appear as a named presence with per-hunk accept/reject

5.12 Extensions

Priority: Phase 2

- Open VSX Registry integration

- Extension API mirroring VS Code's surface (maximises compatibility without Microsoft dependency)
- Extensions run in the isolated Node.js extension host process
- Extension-contributed tabs, commands, language services, and themes

5.13 Productivity Panels

Priority: Phase 1 (notes, todo, diff), Phase 2 (full)

- **Notes tab:** per-project markdown notes, persisted and synced
- **To-do tab:** task list; agents can add tasks automatically when they identify action items
- **Command palette:** `Cmd/Ctrl+K` — fuzzy search over all commands, files, agents, skills, settings
- **Keyboard shortcuts:** fully remappable, importable from VS Code / JetBrains keymaps
- **Diff viewer tab:** standalone file diff outside the editor, for reviewing agent batch changes

6. Cross-Platform Strategy

The frontend UI codebase is shared across all platforms. The sub-service abstraction makes the platform surface irrelevant to the UI layer.

Platform	Shell	Connects to	Feature Set
Desktop (Mac/Win/Linux)	Tauri v2	Local Rust core (in-process) or remote sub-service	Full
Web (browser)	Plain browser tab	Remote sub-service (WebSocket + static bundle served by sub-service)	Full minus panel pop-out / multi-window
Mobile (iOS/Android)	Tauri Mobile	Remote sub-service	Tab-based companion: chat, Orchestrator, HIL, light editing

6.1 Layout Model Per Platform

Desktop:

- Fully tab-based. Every panel is a dockable tab. Tabs can be arranged into resizable split regions.
- Any panel can be popped out into an independent floating window that lives under the main Runyard window (child window). Each floating window supports its own tabs and splits.
- Layout is 100% user-configurable and persisted per workspace.

Web:

- Identical to desktop UI — same components, same tab/split system.
- No panel pop-out or multi-window (browser constraint).

- **Single-instance enforcement:** if a user opens the same sub-service URL in a second browser tab, they see a "Use here" prompt (WhatsApp Web model). Only one active session per browser origin at a time. Enforced via a WebSocket session lock on the sub-service.

Mobile:

- Same visual language and component library as desktop/web.
- No resizable splits or dockable layout — too complex for touch.
- Every panel is a full-screen tab. A persistent bottom tab bar (or swipeable tab strip) lets you switch between: Editor, Explorer, Terminal, Chat, Agent Manager, Orchestrator.
- Single-instance enforcement same as web.

6.2 Appearance & Theming

- **Default typography:** Inter (primary UI font), with Google Sans Flex as display/heading option. Roboto as fallback.
- **Default colour scheme:** Pure black and white base — no greys-for-greys-sake. Light mode and dark mode, toggled per instance.
- **Everything is customisable** from Appearance settings: font family, font size, UI colours (background, surface, accent, text), editor theme, line height, icon set.
- Appearance settings are **local to the instance** — stored in localStorage (web/mobile) or local app config (desktop). They do not sync and are not part of workspace state.
- **Theme format:** compatible with VS Code / OpenVSX theme extensions (Phase 2). In Phase 1, a structured JSON theme file format that maps to CSS custom properties.

6.3 Sub-service Multi-User System

When a sub-service instance is deployed (on a VPS, team server, or any remote machine), it supports multiple users connecting simultaneously. This is managed entirely within the sub-service admin UI.

User management:

- Admin creates users directly in the sub-service settings UI (username + password, or SSO in Phase 2)
- Each user gets assigned filesystem roots they can access (path-level permissions)
- Role-based permission system:
 - `admin` — full access, user management, sub-service config
 - `developer` — read/write access to assigned projects, can run terminals and agents
 - `viewer` — read-only access to assigned projects
 - `agent-only` — API-key-based access for automated agent connections (no UI login)
- Per-project permissions override role defaults

Session management:

- Each connected user has an isolated session (their own agent contexts, conversation history, terminal sessions)
- Admin can see active sessions and terminate them
- Audit log of file operations and agent actions per user

Desktop local mode: user management does not apply when the desktop app connects to its own embedded Rust core (single-user by definition). The multi-user system only activates on sub-service deployments.

Web access flow:

1. User runs sub-service on their machine or VPS
2. Sub-service serves the web frontend at `http://localhost:7820`
3. User reverse-proxies with Nginx/Caddy for public access + TLS (we ship example configs)
4. Connecting users log in with their sub-service credentials
5. They see only their permitted projects and filesystem roots

7. Data & Storage Model

Data Type	Storage	Sync
Workspace files	Native FS (local or remote via sub-service)	Git (user's choice of remote)
Conversation history	SQLite (local per machine)	Optional: synced across machines via sub-service or cloud
Agent configs	YAML/TOML in workspace	Committed to repo if desired
SKILLS	Markdown files in workspace	Committed to repo
Notes / to-dos	Local SQLite	Synced via sub-service
Orchestrator memory	Local SQLite	Synced across machines via sub-service
Remote connections	Local config file	Never synced
Credentials / API keys	OS keychain	Never synced, never in plaintext
Extension data	Local app data dir	Not synced by default

8. Phased Rollout

Phase 0 — Architecture Validation (Pre-build)

- Editor engine benchmark: CodeMirror 6 vs Monaco in a real Tauri webview
- CRDT POC: Yjs in Tauri webview with two connected clients
- Sub-service POC: Rust binary with WebSocket, FS and PTY proxying
- LSP-over-WebSocket POC: LSP client in UI talking to LSP server on sub-service
- Frontend framework decision: Solid.js vs Svelte 5 (benchmark reactivity at IDE-scale)
- IPC schema definition (MessagePack envelope format)
- Name and brand decision

Phase 1 — Core MVP (Desktop, Mac-first)

- Tauridesktop app
- CodeMirror 6 editor: LSP, Tree-sitter, multi-cursor, worktrees
- Tab-based dockable layout (all panels as tabs, detachable)
- PTY terminal
- Sub-service binary: FS, PTY, LSP, Git over WebSocket
- Remote connection UI (connect to sub-service on another machine)
- Chat panel: persistence, context management, cost display, branching
- ACP (Agent Client Protocol) client — editor↔agent
- AgentCP / A2A clients — agent↔agent orchestration
- MCP client + server manager
- Anthropic Skills Protocol
- Agent config UI (global + per-project)
- SKILLS system
- Agent Manager: kanban, parallel management, issue integrations (GitHub + Linear)
- Notes + to-do tabs
- Command palette + keyboard shortcuts
- Git tab (log, staging, worktree management)

Phase 2 — Collaboration, Ecosystem & Mobile

- Yjs real-time collaboration
- Open VSX extension support
- Orchestrator agent (first-party, cross-project)
- ACP agent UI tab (embedded webview)

- Mobile companion app (Tauri Mobile)
- Web version via sub-service static bundle
- Cost dashboards
- Jira + GitLab issue integrations
- SSH bootstrap for sub-service (no pre-install needed)

Phase 3 — Polish & Platform

- Full theme engine (user-created themes)
- Community marketplace for SKILLs and agent configs
- Advanced context strategies (RAG over codebase, local vector search)
- Plugin / extension SDK public release
- Performance hardening pass (profile and optimise the critical path)

9. Open Questions (Decisions Needed)

1. **Editor engine:** CodeMirror 6 vs performance-tuned Monaco? Gated on Phase 0 benchmarks.
2. **Frontend framework:** Solid.js vs Svelte 5? Must commit before Phase 1. Solid's fine-grained reactivity is likely better for complex IDE-scale UIs but Svelte's ergonomics are better for iteration speed.
3. **Sub-service auth model:** simple bearer token (easy to self-host) vs mTLS (harder but more secure)? Initial proposal: bearer token with HTTPS enforced by the user's reverse proxy. mTLS as an optional advanced config.
4. **Orchestrator agent model:** which underlying LLM does the Orchestrator use? User's own API key or a bundled model? Affects cost and offline behaviour.
5. **Cloud sync backend (optional):** for users who want conversation history and notes synced without self-hosting — Supabase (self-hostable) vs a managed service we run? Given open source, Supabase is the right call.
6. **Open source license:** MIT (permissive) vs Apache 2.0 (patent protection) vs AGPL (copyleft, prevents SaaS forks without contribution)? AGPL is worth considering given the self-hostable sub-service model.
7. **Name:** needs to be resolved before Phase 0 completes.

10. Performance Targets (Non-Negotiable)

Metric	Target
Cold startup (app open to usable editor)	<1.5s
File open (10MB file)	<100ms

Metric	Target
Keystroke-to-render latency	<16ms
Base RAM (single workspace, no AI, no LSP)	<150MB
RAM with 3 LSP servers active	<400MB
App binary size	<30MB
Sub-service binary size	<10MB
Sub-service startup time	<500ms
First AI response token (local sub-service)	<1s (excluding model latency)
Remote filesystem operation latency overhead	<20ms over local (LAN)

This document is a living spec. Next action: Phase 0 benchmarks and name decision.